Experiment 'p_4_9_reverse_string' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_4_9_reverse_string
Natural language statement: null
Method signature: p_4_9_reverse_string(s: string) returns (reversed: string)

Ensures

- `|reversed| == |s|`
- `forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_4_9_reverse_string(s: string) returns (reversed: string)

The method should respect the following contract:

ensures |reversed| == |s|, ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1
    ↪ - i]

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnMG6Y79InhlaJbP6ZA03lXLMbptV
Model: gpt-5.5-2026-04-23
Created at: 1780656006
Finish reason: stop
Usage: 308 tokens in, and 3802 tokens out

```
{
  var n := |s|;
  reversed := "";
  var i := n;

  while i > 0
    invariant n == |s|
    invariant 0 <= i <= n
    invariant |reversed| == n - i
    invariant forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k]
    decreases i
  {
    var oldReversed := reversed;
    var oldI := i;
    assert |oldReversed| == n - oldI;
    assert forall k :: 0 <= k < |oldReversed| ==> oldReversed[k] == s[n - 1 - k];

    i := i - 1;
    assert oldI == i + 1;
    assert 0 <= i < n;
```

```
    var piece := s[i..i+1];
    assert |piece| == 1;
    assert piece[0] == s[i];

    reversed := oldReversed + piece;
    assert |reversed| == |oldReversed| + |piece|;
    assert |reversed| == n - i;

    assert forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k] by {
      forall k | 0 <= k < |reversed|
        ensures reversed[k] == s[n - 1 - k]
      {
        if k < |oldReversed| {
          assert reversed[k] == oldReversed[k];
          assert oldReversed[k] == s[n - 1 - k];
        } else {
          assert k >= |oldReversed|;
          assert k < |oldReversed| + |piece|;
          assert |piece| == 1;
          assert k == |oldReversed|;
          assert reversed[k] == piece[0];
          assert piece[0] == s[i];
          assert |oldReversed| == n - oldI;
          assert oldI == i + 1;
          assert |oldReversed| == n - (i + 1);
          assert k == n - (i + 1);
          assert n - 1 - k == i;
          assert reversed[k] == s[n - 1 - k];
        }
      }
    }
  }

  assert |reversed| == |s|;
  assert forall k :: 0 <= k < |s| ==> reversed[k] == s[|s| - 1 - k] by {
    forall k | 0 <= k < |s|
      ensures reversed[k] == s[|s| - 1 - k]
    {
      assert k < |reversed|;
      assert n == |s|;
      assert reversed[k] == s[n - 1 - k];
      assert n - 1 - k == |s| - 1 - k;
    }
  }
}
```

Final Program

```
method p_4_9_reverse_string(s: string) returns (reversed: string)
```

```
      ensures |reversed| == |s|
      ensures forall i :: 0 <= i < |s| ==> reversed[i] == s[|s| - 1 - i]
{
  var n := |s|;
  reversed := "";
  var i := n;

  while i > 0
    invariant n == |s|
    invariant 0 <= i <= n
    invariant |reversed| == n - i
    invariant forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k]
    decreases i
  {
    var oldReversed := reversed;
    var oldI := i;
    assert |oldReversed| == n - oldI;
    assert forall k :: 0 <= k < |oldReversed| ==> oldReversed[k] == s[n - 1 - k];

    i := i - 1;
    assert oldI == i + 1;
    assert 0 <= i < n;

    var piece := s[i..i+1];
    assert |piece| == 1;
    assert piece[0] == s[i];

    reversed := oldReversed + piece;
    assert |reversed| == |oldReversed| + |piece|;
    assert |reversed| == n - i;

    assert forall k :: 0 <= k < |reversed| ==> reversed[k] == s[n - 1 - k] by {
      forall k | 0 <= k < |reversed|
        ensures reversed[k] == s[n - 1 - k]
      {
        if k < |oldReversed| {
          assert reversed[k] == oldReversed[k];
          assert oldReversed[k] == s[n - 1 - k];
        } else {
          assert k >= |oldReversed|;
          assert k < |oldReversed| + |piece|;
          assert |piece| == 1;
          assert k == |oldReversed|;
          assert reversed[k] == piece[0];
          assert piece[0] == s[i];
          assert |oldReversed| == n - oldI;
          assert oldI == i + 1;
          assert |oldReversed| == n - (i + 1);
          assert k == n - (i + 1);
          assert n - 1 - k == i;
          assert reversed[k] == s[n - 1 - k];
```

```
      }
    }
  }
 }

 assert |reversed| == |s|;
 assert forall k :: 0 <= k < |s| ==> reversed[k] == s[|s| - 1 - k] by {
  forall k | 0 <= k < |s|
    ensures reversed[k] == s[|s| - 1 - k]
  {
    assert k < |reversed|;
    assert n == |s|;
    assert reversed[k] == s[n - 1 - k];
    assert n - 1 - k == |s| - 1 - k;
  }
 }
}
```

Total Token Usage

Input tokens: 308
Output tokens: 3802
Reasoning tokens: 3072
Sum of 'total tokens': 4110

Experiment Timings

Overall Experiment started at 1780656006396, ended at 1780656102777, lasting 96381ms (96.38 seconds)
Iteration #1 started at 1780656006396, ended at 1780656102777, lasting 96381ms (96.38 seconds)